

BAB III

LANDASAN TEORI

3.1 Rancang Bangun

Rancang bangun sangat berkaitan dengan perancangan sistem yang merupakan menciptakan dan membuat suatu aplikasi ataupun satu kesatuan untuk merancang dan membangun sebuah aplikasi yang belum ada pada suatu instansi atau objek tersebut (Andri Syahputra, 2022). Menurut KBBI (Kamus Besar Bahasa Indonesia), kata “rancang” merupakan kata dasar dari “merancang” yang berarti mengatur segala suatu (sebelum bertindak, mengerjakan, atau melakukan sesuatu) atau merencanakan.

Menurut Jogianto dikutip oleh (Mulyanto, 2020), dalam Jurnal JINTEKS Vol. 2 No. 1 (2020), Rancang bangun (desain) adalah tahap dari setelah analisis dari siklus pengembangan sistem yang merupakan pendefinisian dari kebutuhan-kebutuhan fungsional, serta menggambarkan bagaimana suatu sistem dibentuk yang dapat berupa penggambaran, perencanaan dan pembuatan sketsa atau pengaturan dari beberapa elemen yang terpisah ke dalam satu kesatuan yang utuh dan berfungsi, termasuk menyangkut mengkonfigurasi dari komponen-komponen perangkat lunak dari suatu sistem.

Dari penjelasan diatas dapat disimpulkan rancang bangun sistem merupakan kegiatan menterjemahkan hasil analisa kedalam bentuk paket perangkat lunak kemudian menciptakan sistem tersebut atau memperbaiki sistem yang ada.

3.2 Pengertian Sistem

Pengertian sistem sangatlah luas dan mempengaruhi semua aspek kehidupan. Sistem sangat diperlukan dalam melakukan kinerja yang baik dan terstruktur terhadap manajemen. Keterpaduan sistem ini memungkinkan terciptanya kerjasama untuk menghasilkan informasi yang cepat, tepat dan akurat. Sistem dapat didefinisikan dengan 2 (dua) pendekatan, yaitu sistem

yang menekankan pada prosedur dan sistem yang menekankan pada elemen komponennya.

Sistem yang menekankan pada prosedur, menurut Jogiyanto dikutip oleh (Mulyanto, 2020), dalam Jurnal JINTEKS Vol. 2 No. 1 (2020), "Sistem adalah suatu jaringan kerja dari prosedur-prosedur yang saling berhubungan, berkumpul bersama-sama untuk melakukan suatu kegiatan atau penyelesaian suatu sasaran tertentu". Sedangkan sistem yang menekankan pada elemen yaitu: "Sistem adalah suatu seri dari komponen-komponen yang saling berhubungan, bekerja sama didalam suatu kerangka kerja tahapan yang terpadu untuk menyelesaikan, mencapai sasaran yang telah ditetapkan sebelumnya".

Berdasarkan definisi di atas, dapat ditarik kesimpulan bahwa sistem adalah suatu kumpulan atau jaringan kerja yang saling berhubungan dan saling ketergantungan satu sama lain untuk sama-sama menyelesaikan sasaran yang diteliti atau tujuan.

3.3 Website

Website Menurut Yuhefizar dikutip oleh (Prayitno, 2018) dalam jurnal IJSE Vol. 1 No. 1 (2018), pengertian website adalah "keseluruhan halaman halaman *web* yang terdapat dari sebuah domain yang mengandung informasi".

Menurut Yuhefizar dikutip oleh (Prayitno, 2018) dalam jurnal IJSE Vol. 1 No. 1 (2018), sejak awal 1990, *world wide web* atau *website* merevolusi kehidupan pribadi maupun *professional*. *Web* menjadi situs yang terus berkembang dan sebagai perpustakaan informasi yang ada di mana-mana yang dapat diakses melalui mesin pencari dan *portal*. *Web* menjadi tempat penyimpanan media yang memfasilitasi hosting dan berbagi sumber daya yang sering kali gratis dan sebagai pendukung layanan *do-it-yourself*. *Web* juga menjadi *platform* perdagangan tempat orang dan perusahaan semakin menjalankan bisnisnya.

3.4 Metode *Rational Unified Process* (RUP)

Rational Unified Process (RUP) merupakan suatu metode rekayasa perangkat lunak yang dikembangkan dengan mengumpulkan berbagai *best*

practises yang terdapat dalam industri pengembangan perangkat lunak. Ciri utama metode ini adalah menggunakan *use-case driven* dan pendekatan iteratif untuk siklus pengembangan perangkat lunak (Taryana, 2020).

3.5 Codeigniter

Codeigniter merupakan sebuah *framework* aplikasi yang akan dibangun berbasis *web* yang menggunakan konsep MVC (*Model, View, Controller*). *Framework* PHP ini dapat menjadi tools bagi seorang *web developer* untuk mengembangkan suatu *situs* dengan lebih mudah karena menyediakan *resource* yang lengkap (Setiawansyah, 2020).

3.6 WAMP

WAMP berfungsi sebagai server virtual/lokal yang berjalan di komputer Anda. Perangkat lunak ini digunakan untuk menguji semua fitur *WordPress* tanpa mengkhawatirkan risiko atau masalah di situs web online. Ini mungkin karena "situs web" sama sekali tidak terhubung ke internet dan baru saja diinstal di komputer Anda. Memiliki WAMP akan membuat pekerjaan pengembang *back-end* dan *front-end* jauh lebih mudah. Karena mereka dapat mendesain dan mengembangkan sebuah website tanpa harus khawatir mengubah kode pada website (Nawadwipa, 2020).

3.7 Bahasa Pemrograman

Bahasa pemrograman adalah untuk mengembangkan proses pembuatan website penulis menggunakan Bahasa pemrograman diantaranya, adalah :

3.7.1 HTML

Perkembangan internet ini, diikuti pula dengan perkembangan HTML (*Hyper Text Markup Language*) yang merupakan bahasa standar yang paling umum atau yang paling sering digunakan oleh para *web programmer* dalam membangun sebuah aplikasi web (Gumolung, 2020). Kata Hypertext dari HTML menekankan pengertian: text yang lebih dari sekedar teks ('*hyper*'- text). Maksudnya selain berfungsi sebagai teks

biasa, sebuah teks di HTML juga bisa berfungsi sebagai penghubung ke halaman lain atau dikenal dengan istilah link. Nantinya kita juga akan melihat bahwa tidak hanya teks saja yang bisa digunakan sebagai link, tetapi bisa berupa gambar. Link inilah yang menjadi inti dari HTML.(Pratama, 2020)

3.7.2 PHP

PHP singkatan dari PHP yaitu Hypertext Preprocessor. PHP ini merupakan suatu bahasa pemrograman server-side yang dirancang untuk pengembangan web. PHP dikatakan server-side lantaran program yang diberikan kan dijalankan atau diproses pada komputer yang bertindak sebagai server. Contohnya saat pengguna mengakses suatu situs maka web browser akan melakukan request ke serverHypertext Preprocessor (Dahlan et al., 2020).

Sebagai sebuah aplikasi, website tersebut hendaknya memiliki sifat dinamis dan interaktif. Memiliki sifat dinamis artinya, website tersebut bisa berupa tampilan kontennya sesuai, kondisi tertentu (misalnya menampilkan produk yang berbeda-beda untuk setiap pengunjung). Interaktif artinya, website tersebut dapat member feedback bagi user (misalnya, menampilkan hasil pencarian produk). PHP merupakan bahasa pemrograman berjenis server-side. Dengan demikian, PHP akan diproses oleh server yang hasil olahannya akan dikirim kembali ke browser. Oleh karena itu, salah-satu tool yang harus tersedia sebelum memulai pemrograman PHP adalah server (Hidayat, 2019).

3.7.3 MYSQL

MySQL yaitu sebuah program *database server* yang mampu menerima dan mengirimkan datanya dengan sangat cepat, *multi user* serta menggunakan perintah standar SQL (*Structured Query Language*). MySQL bersifat *Open Source* yang dapat dijalankan diberbagai *platform* misalnya Windows, Linux, dan lain sebagainya (Musril, 2021).

MySQL adalah produk DBMS open source yang berjalan pada UNIX, Linux, dan Windows. Sumber dan kode biner MySQL dapat didownload dari situs Web MySQL (<http://www.mysql.com>). Keterbatasan MySQL tidak mendukung View, prosedur tersimpan, maupun trigger. Akan tetapi, semua hal tersebut ada pada to-do-list MySQL, sehingga periksa dokumentasi terakhir untuk menentukan apakah beberapa fitur-fitur tersebut telah ditambahkan ke produk tersebut pada realease-realease yang terbaru (Sahi, 2020).

3.8 Alat Bantu Perancangan Sistem

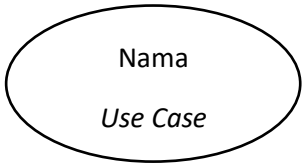
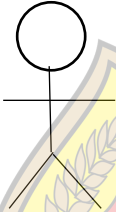
Alat bantu perancangan sistem merupakan alat yang umumnya digunakan untuk membantu analisis dalam membuat model alur jalannya sebuah sistem. Alat bantu perancangan sistem yang akan digunakan dalam penyusunan Tugas Akhir ini adalah *Unified Modelling Language (UML)*.

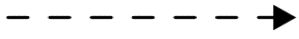
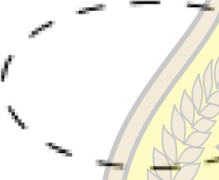
UML adalah salah satu standar bahasa yang banyak digunakan di dunia industri untuk mendefinisikan *requirement*, membuat analisis dan desain, serta menggambarkan arsitektur dalam pemrograman berorientasi objek (Adriani, 2019). Berikut *Diagram Unified Modeling Language (UML)* antara lain sebagai berikut :

3.8.1 Use case Diagram

Use case Diagram merupakan pemodelan untuk kelakuan sistem informasi yang akan dibuat. *Use case* bekerja dengan mendeskripsikan tipikal interaksi antara *user* sebuah sistem dengan sistemnya sendiri melalui sebuah cerita bagaimana sistem itu dipakai (Adriani, 2019). Berikut adalah tabel simbol-simbol dari *Use case Diagram*:

Tabel 1. 1 Use case Diagram

Simbol	Deskripsi
<i>Use case</i> 	Fungsionalitas yang disediakan sistem sebagai <i>unit-unit</i> yang saling bertukar peran antar unit atau <i>actor</i>, biasanya dinyatakan dengan menggunakan kata kerja diawal frase nama <i>use case</i>.
<i>Actor</i> 	Orang, proses, atau sistem lain yang berinteraksi dengan sistem informasi yang akan dibuat diluar sistem informasi yang akan dibuat itu sendiri, jadi walaupun simbol dari <i>actor</i> adalah gambar orang, tapi <i>actor</i> belum tentu merupakan orang, biasanya dinyatakan kata benda di awal frase nama <i>actor</i>.
Asosiasi/Association 	Komunikasi antara <i>actor</i> dan <i>use case</i> yang berpartisipasi pada <i>use case</i> yang memiliki interaksi dengan <i>actor</i>
Ekstensi/Extend <<Extend>> 	Relasi <i>use case</i> tambahan ke sebuah <i>use case</i> dimana <i>use case</i> yang ditambahkan dapat berdiri sendiri walau tanpa <i>use case</i> tambahan itu, mirip dengan prinsip inheritance pada pemrograman berorientasi objek, biasanya <i>use case</i> tambahan memiliki nama depan yang sama dengan <i>use case</i> yang ditambahkan.

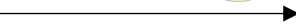
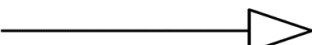
Dependency 	Hubungan dimana perubahan yang terjadi pada suatu elemen mandiri akan mempengaruhi elemen yang bergantung pada elemen yang tidak mandiri.
Include 	Menspesifikasikan bahwa <i>use case</i> sumber eksplisit.
Collaboration 	Interaksi aturan-aturan dan elemen lain yang bekerja sama untuk menyediakan perilaku yang lebih besar dari jumlah elemen-elemen (sinergi).
Note 	Elemen fisik yang eksis saat aplikasi dijalankan dan mencerminkan suatu sumber daya komputasi.

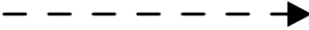
USM

3.8.2 Class diagram

Class diagram merupakan gambaran struktur sistem dari segi pendefinisian kelas-kelas yang akan dibuat untuk membangun sistem. *Class diagram* terdiri dari atribut dan operasi dengan tujuan pembuat program dapat membuat hubungan antara dokumentasi dan perangkat sesuai (Adriani, 2019). Berikut adalah 19 symbol-simbol dari *Class diagram* :

Tabel 1. 2 *Class diagram*

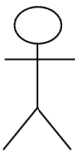
Simbol	Deskripsi
<i>Class</i> 	Kelas pada struktur sistem
<i>Antarmuka/Interface</i> 	Sama dengan konsep <i>interface</i> dalam pemrograman berorientasi objek.
<i>Asosiasi/Association</i> 	Relasi antar kelas dengan makna umum, asosiasi biasanya juga disertai dengan <i>multiplicity</i> .
Asosiasi berarah/<i>Direct Association</i> 	Relasi antar kelas dengan makna kelas yang satu digunakan oleh kelas yang lain, asosiasi biasanya juga disertai dengan <i>multiplicity</i> .
Generalisasi 	Relasi antar kelas dengan makna generalisasi- spesialisasi (umum-khusus).
<i>Agregasi/Aggregation</i> 	Relasi antar kelas dengan makna semua bagian (<i>whole- part</i>).

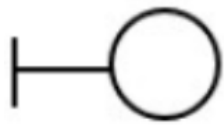
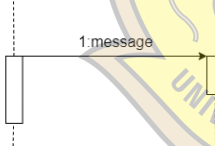
Ketergantungan/Dependency 	Hubungan dimana perubahan yang terjadi pada suatu elemen mandiri (<i>independent</i>) akan mempengaruhi elemen yang bergantung pada elemen yang tidak mandiri.
--	--

3.8.3 Sequence Diagram

Sequence Diagram menggambarkan kelakuan objek pada *use case* dengan mendeskripsikan waktu hidup objek dan pesan yang dikirimkan dan diterima antar objek. Gambaran *sequence diagram* dibuat minimal sebanyak pendefinisian *use case* yang memiliki proses sendiri atau yang penting semua *use case* yang telah didefinisikan interaksi jalannya pesan sudah dicakup pada *sequence diagram* sehingga semakin banyak *use case* yang didefinisikan, maka *sequence diagram* yang dibuat harus juga semakin banyak (Adriani, 2019). Berikut adalah simbol-simbol dari *Sequence Diagram* :

Tabel 1. 3 *Sequence Diagram*

Simbol	Deskripsi
<i>Actor</i> 	Orang ataupun pihak yang akan mengelola sistem.
Garis Hidup/<i>Lifeline</i> 	Menggambarkan sebuah objek dalam sebuah sistem atau salah satu komponennya.

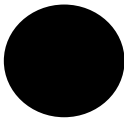
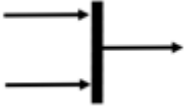
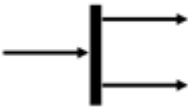
<i>Boundary</i> 	Pemodelan bagian dari sistem yang bergantung pada pihak lain disekitarnya dan merupakan pembatasan sistem dengan dunia luar.
<i>Control</i> 	Pemodelan “perilaku mengatur” khusus untuk satu atau beberapa <i>use case</i> saja.
<i>Entity</i> 	Permodelan informasi yang harus disimpan oleh sistem yang memperlihatkan struktur data dari suatu sistem.
<i>Message</i> 	Mengidentifikasi urutan komunikasi yang terjadi antar objek.

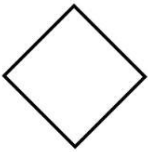
3.8.4 Activity Diagram

USM

Menurut (Putra & Andriani, 2019) *Activity Diagram* merupakan diagram yang menggambarkan *workflow* atau aktivitas dari sebuah sistem yang ada pada perangkat lunak. Berikut adalah tabel simbol-simbol dari *Activity Diagram* :

Tabel 1. 4 Activity Diagram

Simbol	Deskripsi
	Titik awal atau permulaan.
	Titik akhir atau akhir dari aktivitas.
	Activity atau aktivitas yang dilakukan oleh actor.
	State dari sistem yang mencerminkan eksekusi arti sistem.
	Untuk menggabungkan beberapa kegiatan secara <i>parallel</i> menjadi satu.
	Sebuah kejadian yang memicu sebuah <i>state</i> objek dengan cara memperbaharui satu atau lebih nilai atributnya.
	Menunjukkan kegiatan yang dilakukan secara <i>parallel</i> .

<i>Decision</i> 	Percabangan dimana ada pilihan aktivitas yang lebih dari satu
--	---

3.8.5 Pengujian *White Box*

Pengujian *White Box*, adalah suatu metode pengujian aplikasi yang menggunakan penjelasan struktur kontrol sebagai bagian dari *component level design* untuk membuat test cases. *White Box* sendiri mempunyai beberapa teknik di dalam pengujiannya, seperti : *Data Flow Testing*, *Control Flow Testing*, *Basic Path / Path Testing*, dan *Loop Testing*. Dalam Pengujian *White Box* para penguji perlu mengetahui secara dalam *source code* yang akan diuji. Pengujian *White Box* dapat mengungkap kesalahan implementasi dari sebuah aplikasi. Pengujian ini dapat diterapkan pada tingkatan integrasi, unit dan sistem (Farhan, 2021).

3.8.6 Pengujian *Black Box*

Black box testing merupakan pengujian kualitas perangkat lunak yang berfokus pada fungsionalitas perangkat lunak. Pengujian *black box* bertujuan untuk menemukan fungsi yang tidak benar, kesalahan antarmuka, kesalahan pada struktur data, kesalahan performansi, kesalahan inisialisasi dan terminasi (Yahya, 2021).